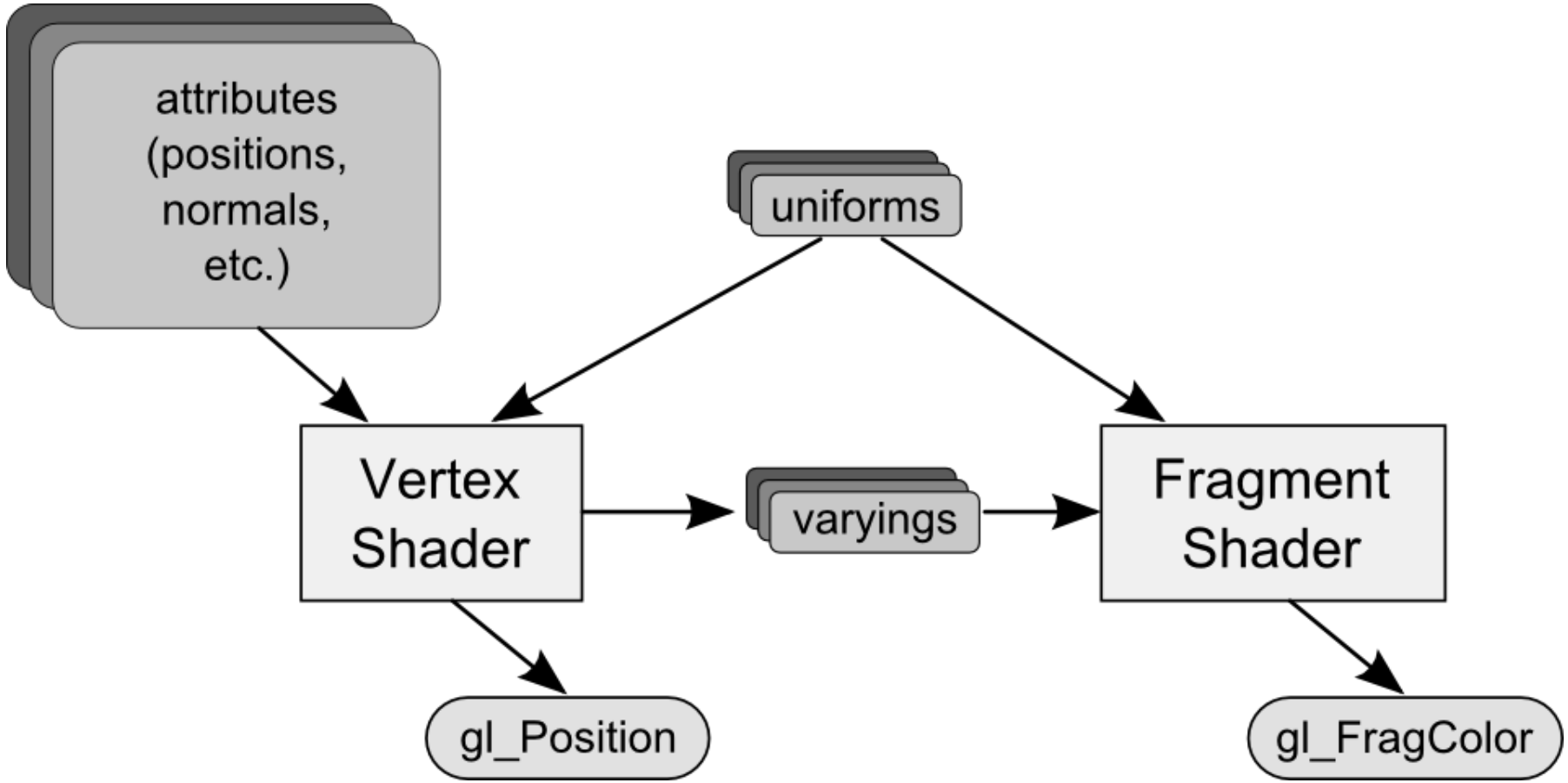
Shader

- Historisch:
  - Shader = Programmkode, der berechnet, wie Teile der Szene eingefärbt werden.
- Heute:
  - Shader = Programmkode, der auf der Grafikkarte ausgeführt wird und fast alle Eigenschaften einer Szene verändern kann.

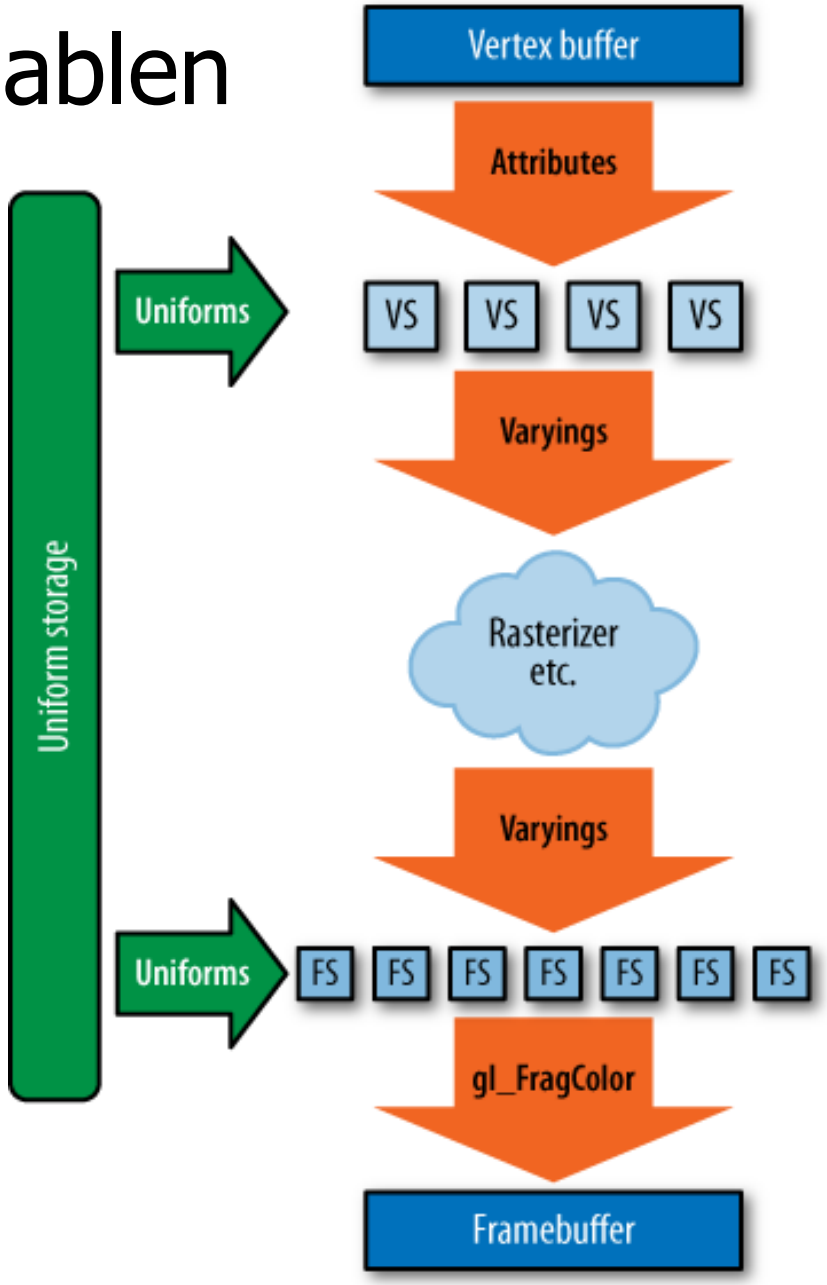
SHADER IN THREE.JS

Shader in WebGL bzw. Three.js



Shader: Arten von Variablen

- **Uniforme Variablen** enthalten Werte, die während der ganzen Berechnung eines Frames **konstant** bleiben, z.B. die Position eines Objektes. Vertex und Fragment Shader haben uniforme Variablen.
- **Attribute** sind Werte, die zu einzelnen Eckpunkten (Vertex) gehören, z.B. die Farbe eines Punktes. **Nur Vertex Shader** haben Attribute.
- **Varying** Variablen werden verwendet, um **Werte mit dem Fragment Shader zu teilen**. Dazu müssen die varying Variablen mit dem gleichen Typ und Namen in beiden Shadern deklariert werden. Beispiel: Die Normale eines Eckpunktes kann zur Beleuchtungsberechnung im Fragment Shader verwendet werden.



Online-Quellen zu Shader in Three.js und zur GLSL

- https://threejs.org/docs/#api/en/materials/ShaderMaterial
- https://threejs.org/docs/#api/en/renderers/webgl/WebGLProgram

- http://code.tutsplus.com/tutorials/webgl-with-threejs-shaders--net-36054

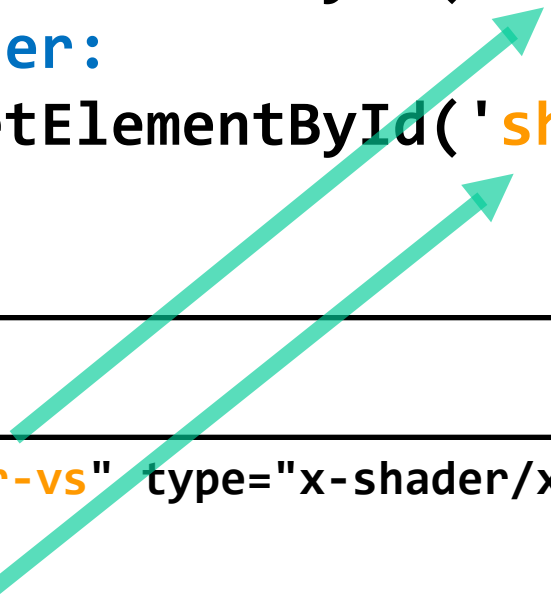
- GLSL
  - http://www.shaderific.com/glsl.html

ShaderMaterial

Shader1.html

```
var material = new THREE.ShaderMaterial({
 vertexColors: THREE.VertexColors,
 vertexShader:
  document.getElementById('shader-vs').textContent,
 fragmentShader:
  document.getElementById('shader-fs').textContent
 });
```

```
<script id="shader-vs" type="x-shader/x-fragment">
  …
<script id="shader-fs" type="x-shader/x-vertex">
  …
```



Die Shader

Shader1.html

```
<script id="shader-fs" type="x-shader/x-fragment">
  varying vec3 vColor;
  void main(void) {
      gl_FragColor =
        vec4(vColor.x, vColor.y, vColor.z, 1.0);
  }
</script>

<script id="shader-vs" type="x-shader/x-vertex">
  varying vec3 vColor;
   void main(void) {
    gl_Position = projectionMatrix
                  *modelViewMatrix
                  *vec4(position, 1.0);
    vColor= color;
  }
 </script>
```

Fragment Shader

Vertex Shader

Built-in uniforms

Erst transformieren, dann projizieren.

Umwandlung in homogene Koordinaten

Built-in attribute

ShaderMaterial

Selbst definierte Uniforms

Shader2.html

```
var myUniforms = {
      myColor: { type: "c",
                 value: new THREE.Color(0xabcdef) }
      };
var material = new THREE.ShaderMaterial({
  uniforms: myUniforms,
  vertexShader:
   document.getElementById('shader-vs').textContent,
  fragmentShader:
   document.getElementById('shader-fs').textContent
 });
```

Die Shader

Shader2.html

```
<script id="shader-fs" type="x-shader/x-fragment">
   uniform vec3 myColor;

   void main(void) {
      gl_FragColor =
       vec4(myColor.x, myColor.y, myColor.z, 1.0);
   }
</script>

<script id="shader-vs" type="x-shader/x-vertex">

  void main(void) {
    gl_Position = projectionMatrix
                  *modelViewMatrix
                  *vec4(position, 1.0);
   }
</script>
```

Selbst definiertes Uniform

ShaderMaterial

Shader3.html

```
var material = new THREE.ShaderMaterial({
  vertexShader:
   document.getElementById('shader-vs').textContent,
  fragmentShader:
   document.getElementById('shader-fs').textContent
 });
```

Die Shader

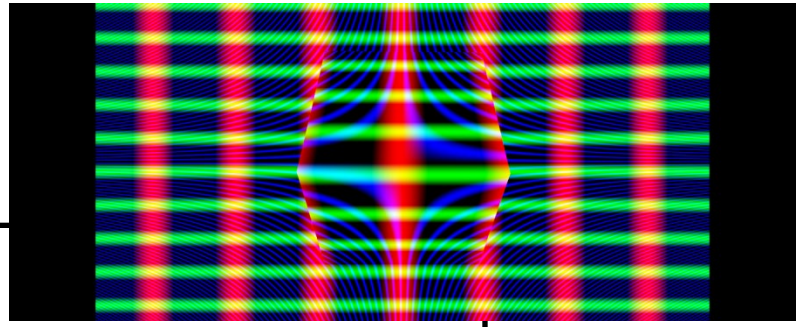
Shader3.html

```
<script id="shader-fs" type="x-shader/x-fragment">
 varying vec3 myPosColor;
 void main(void) {
   gl_FragColor =
    vec4(myPosColor.x, myPosColor.y, myPosColor.z, 1.0);
   }
</script>

<script id="shader-vs" type="x-shader/x-vertex">
varying vec3 myPosColor;
 void main(void) {
   gl_Position = projectionMatrix*modelViewMatrix
                      *vec4(position, 1.0);
  //myPosColor=position;
  myPosColor=
    vec3(gl_Position.x, gl_Position.y, 0.0);
    //vec3(position.x, position.y, position.z);
   }
 </script>
```

Selbst definiertes Varying

Aufgabe



https://t.ly/B2qsd
https://www.st.uni-trier.de/lectures/S26/CG/webgl/shader/exercise1ace.html

- Es gibt die Funktionen **sin()** und **cos()** im Shader.
- Fügen Sie eine uniform Variable **time** vom Typ float ("f") hinzu, die Sie während der Animation schrittweise erhöhen:
  - material.uniforms.time.value += 0.1;

Hinweis:
3.0 ungleich 3

- Ändern Sie Eigenschaften im Shader in Abhängigkeit von der Variablen **time**:
  - Eigenschaften, die zur Farbbestimmung verwendet werden.
  - Eigenschaften, die zur Transformation bzw. Projektion verwendet werden.

```
<script id="cube-shader-fs" type="x-shader/x-fragment">
 varying vec3 posLocal;
 varying vec4 pos3D;
 varying vec4 pos2D;
 uniform float aspectRatio;

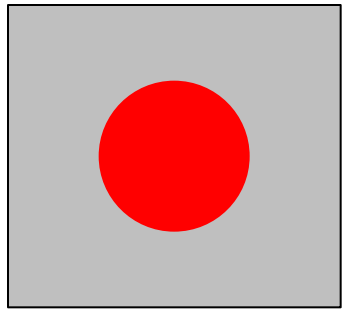
 varying vec3 vertex_normal;

 void main(void) {
   float x=posLocal.x;
   float y=posLocal.y;
   float z=posLocal.z;

   vec4 color = vec4(0.7, 0.7, 0.6, 1.0);
   vec3 light = normalize(vec3(0.0,5.0,5.0));

   // Lambert lighting model
    vec3 col3=max(0.0,dot(vertex_normal,light ))*color.rgb;
    gl_FragColor = vec4(col3.r, col3.g, col3.b, 1.0);

 }
</script>
```



https://t.ly/B2qsd

https://www.st.uni-trier.de/lectures/S26/CG/webgl/shader/exercise1ace.html